Programação II
AULA 02 – Modularização ou Função

1 Função

Funções são as estruturas que permitem ao usuário separar seus programas em blocos. Se não as tivéssemos, os programas teriam que ser curtos e de pequena complexidade. Para fazermos programas grandes e complexos temos de construí-los bloco a bloco.

Sintaxe de uma função:

```
tipo_de_retorno nome_da_função (declaração_de_parâmetros)
{
          corpo_da_função
}
```

O tipo-de-retorno é o tipo de variável que a função vai retornar.

A declaração de parâmetros é uma lista com a seguinte forma geral: tipo var1, tipo var2, ... , tipo varN

O tipo deve ser especificado para cada uma das N variáveis de entrada. É na declaração de parâmetros que informamos ao compilador quais serão as entradas da função.

O corpo da função é onde as entradas são processadas, saídas são geradas ou outras coisas são feitas.

1.1 O Comando return

O comando return tem a seguinte forma geral:

```
return valor_de_retorno;  ou  return;
```

Digamos que uma função está sendo executada. Quando se chega a uma declaração return a função é encerrada imediatamente e, se o valor de retorno é informado, a função retorna este valor. É importante lembrar que o valor de retorno fornecido tem que ser compatível com o tipo de retorno declarado para a função.

Uma função pode ter mais de uma declaração return. Isto se torna claro quando pensamos que a função é terminada quando o programa chega à primeira declaração return. Abaixo estão dois exemplos de uso do return:

Exemplo 01:

```
#include <stdio.h>
#include <conio.h>

int Quadrado (int a)
{
      return (a*a);
```

```
}                  //fim da função

int main ()
{
      int num;

      printf ("Entre com um numero: ");
      scanf ("%d",&num);
      num=Quadrado(num);
      printf ("\n\nO seu quadrado vale: %d\n",num);

      return 0;
}                  //fim do programa pincipal
```

Exemplo 02:

```
#include <stdio.h>
#include <conio.h>

int EPar (int a)
{
      if (a%2)           /* Verifica se a e divisivel por dois */
          return 0;        /* Retorna 0 se nao for divisivel */
      else
          return 1;        /* Retorna 1 se for divisivel */
}                     // fim da função

int main ()
{
      int num;

      printf ("Entre com numero: ");
      scanf ("%d",&num);
      if (EPar(num))
            printf ("\n\nO numero e par.\n");
      else
            printf ("\n\nO numero e impar.\n");
      getch();

      return 0;
}
```

É importante notar que, como as funções retornam valores, podemos aproveitá-los para fazer atribuições, ou mesmo para que estes valores participem de expressões. Mas não podemos fazer:

```
func(a,b)=x;  /* Errado! */
```

No segundo exemplo vemos o uso de mais de um return em uma função.

1.2 Protótipos de Funções

Até agora escrevemos as funções antes de escrevermos a função main(). Isto é, as funções estão fisicamente antes da função main(). Isto foi feito por uma razão. Imagine-se na pele do compilador. Se você fosse compilar a função main(), onde são chamadas as funções, você teria que saber com antecedência quais são os tipos de retorno e quais são os parâmetros das funções para que você pudesse gerar o código corretamente. Por isso as funções foram colocadas antes da função main(): quando o compilador chegasse à função main() ele já teria compilado as funções e já saberia seus formatos.

Muitas vezes teremos o nosso programa espalhado por vários arquivos. Ou seja, estaremos chamando funções em um arquivo que serão compiladas em outro arquivo.

A solução são os protótipos de funções. Protótipos são declarações de funções. Isto é, você declara uma função que irá usar. O compilador toma então conhecimento do formato daquela função antes de compilá-la.

Um protótipo tem o seguinte formato:

```
tipo_de_retorno nome_da_função (declaração_de_parâmetros);
```

onde o tipo-de-retorno, o nome-da-função e a declaração-de-parâmetros são os mesmos que você pretende usar quando realmente escrever a função. Repare que os protótipos têm uma nítida semelhança com as declarações de variáveis. Vamos implementar agora um dos exemplos anterior com algumas alterações e com protótipos:

```
#include <stdio.h>
float Quadrado (float a);

int main ()
{
      float num;
      printf ("Entre com um numero: ");
      scanf ("%f",&num);
      num= Quadrado (num);
      printf ("\n\nO seu quadrado vale: %f\n",num);
      getch();
      return 0;
}

float Quadrado (float a)
{
      return (a*a);
}
```

Observe que a função Square() está colocada depois de main(), mas o seu protótipo está antes. Sem isto este programa não funcionaria corretamente.

Usando protótipos você pode construir funções que retornam quaisquer tipos de variáveis. Os protótipos não só ajudam o compilador. Eles ajudam a você também: usando protótipos, o compilador evita erros, não deixando que o programador use funções com os parâmetros errados e com o tipo de retorno errado.

1.3 O Tipo void

Em inglês, void quer dizer vazio.. Ele nos permite fazer funções que não retornam nada e funções que não têm parâmetros. Podemos agora escrever o protótipo de uma função que não retorna nada:

```
void nome_da_função (declaração_de_parâmetros);
```

Neste tipo de função, não temos valor de retorno na declaração return. Aliás, neste caso, o comando return não é necessário na função.

Podemos, também, fazer funções que não têm parâmetros:

```
tipo_de_retorno nome_da_função (void);
```

ou, ainda, que não tem parâmetros e não retornam nada:

```
void nome_da_função (void);
```

Um exemplo de funções que usam o tipo void:

```
#include <stdio.h>
#include <conio.h>

void Mensagem (void);

int main ()
{
      Mensagem();
      printf ("\tDiga de novo:\n");
      Mensagem();
      return 0;
}

void Mensagem (void)
{
      printf ("Ola! Eu estou vivo.\n");
}
```

Se quisermos que a função retorne algo, devemos usar a declaração return. Se não quisermos, basta declarar a função como tendo tipo-de-retorno void. Devemos lembrar agora que a função main() é uma função e como tal devemos tratá-la. O compilador acha que a função main() deve retornar um inteiro. Isto pode ser interessante se quisermos que o sistema operacional receba um valor de retorno da função main(). Se o programa retornar zero, significa que ele terminou normalmente, e, se o programa retornar um valor diferente de zero, significa que o programa teve um término anormal.

1.4 Variáveis locais

Estas são aquelas que só têm validade dentro do bloco no qual são declaradas. Podemos declarar variáveis dentro de qualquer bloco. Só para lembrar: um bloco começa quando abrimos uma chave e termina quando fechamos a chave. Por exemplo, um comando for pode ter variáveis locais e que não serão conhecidas fora dali. A declaração de variáveis locais é a primeira coisa que devemos colocar num bloco. A característica que torna as variáveis locais tão importantes é justamente a de serem exclusivas do bloco. Podemos ter quantos blocos quisermos com uma variável local chamada x, por exemplo, e elas não apresentarão conflito entre elas.

Abaixo vemos um exemplo de variáveis locais:

```
func1 (...)
{
      int abc,x;
      ...
}
func (...)
{
      int abc;
      ...
}
void main ()
{
      int a,x,y;
      for (...)
        {
            float a,b,c;
        ...
        }
      ...
}
```

No programa acima temos três funções. As variáveis locais de cada uma delas não irão interferir com as variáveis locais de outras funções. Assim, a variável abc de func1() não tem nada a ver (e pode ser tratada independentemente) com a variável abc de func2(). A variável x de func1() é também completamente independente da variável x de main(). As variáveis a, b e c são locais ao bloco for. Isto quer dizer que só são conhecidas dentro deste bloco for e são desconhecidas no resto da função main(). Quando usarmos a variável a dentro do bloco for estaremos usando a variável a local ao for e não a variável a da função main().

1.5 Parâmetros formais

O segundo tipo de variável que veremos são os parâmetros formais. Estes são declarados como sendo as entradas de uma função. O parâmetro formal é uma variável local da função. Você pode também alterar o valor de um parâmetro formal, pois esta alteração não terá efeito na variável que foi passada à função. Isto tem sentido, pois quando o C passa parâmetros para uma função, são passadas apenas cópias das variáveis. Isto é, os parâmetros formais existem independentemente das variáveis que foram passadas para a função. Eles tomam apenas uma cópia dos valores passados para a função.

1.6 Variáveis globais

Variáveis globais são declaradas fora de todas as funções do programa. Elas são conhecidas e podem ser alteradas por todas as funções do programa. Quando uma função tem uma variável local com o mesmo nome de uma variável global a função dará preferência à variável local. Vamos ver um exemplo:

```
int z,k;
func1 (...)
{
      int x,y;
      ...
}

func2 (...)
{
      int x,y,z;
```

```
        ...
        z=10;
        ...
}

main ()
{
        int count;
...
}
```

No exemplo acima as variáveis z e k são globais. Veja que func2() tem uma variável local chamada z. Quando temos então, em func2(), o comando z=10 quem recebe o valor de 10 é a variável local, não afetando o valor da variável global z.

Evite ao máximo o uso de variáveis globais. Elas ocupam memória o tempo todo (as locais só ocupam memória enquanto estão sendo usadas) e tornam o programa mais difícil de ser entendido e menos geral.

Exercício: Analise o seguinte programa e aponte o valor de cada variável sempre que solicitado:

```
#include <stdio.h>
int num;
int func(int a, int b)
{
        a = (a+b)/2;  /* Qual e o valor de a apos a atribuicao? */
        num -= a;
        return a;
}
int main()
{
        int first = 0, sec = 50;
        num = 10;
        num += func(first, sec);  /* Qual e o valor de num, first e sec */
                                  /* antes e depois da atribuicao?      */
        printf("\n\nConfira! num = %d\tfirst = %d\tsec = %d",num, first, sec);
        return 0;
}
```

1.7 Passagem de parâmetros por valor e passagem por referência

Vimos que quando chamamos uma função os parâmetros formais da função copiam os valores dos parâmetros que são passados para a função. Isto quer dizer que não são alterados os valores que os parâmetros têm fora da função. Este tipo de chamada de função é denominado chamada por valor. Isto ocorre porque são passados para a função apenas os valores dos parâmetros e não os próprios parâmetros. Veja o exemplo abaixo:

```
#include <stdio.h>
float sqr (float num);
void main ()
{
        float num,sq;
        printf ("Entre com um numero: ");
        scanf ("%f",&num);
        sq=sqr(num);
        printf ("\n\nO numero original e: %f\n",num);
        printf ("O seu quadrado vale: %f\n",sq);
}
float sqr (float num)
```

```
{
      num=num*num;
      return num;
}
```

No exemplo acima o parâmetro formal num da função sqr() sofre alterações dentro da função, mas a variável num da função main() permanece inalterada: é uma chamada por valor.

Outro tipo de passagem de parâmetros para uma função ocorre quando alterações nos parâmetros formais, dentro da função, alteram os valores dos parâmetros que foram passados para a função. Este tipo de chamada de função tem o nome de chamada por referência. Este nome vem do fato de que, neste tipo de chamada, não se passa para a função os valores das variáveis, mas sim suas referências (a função usa as referências para alterar os valores das variáveis fora da função).

O C só faz chamadas por valor. Isto é bom quando queremos usar os parâmetros formais à vontade dentro da função, sem termos que nos preocupar em estar alterando os valores dos parâmetros que foram passados para a função. Mas isto também pode ser ruim às vezes, porque podemos querer mudar os valores dos parâmetros fora da função também. Há entretanto, no C, um recurso de programação que podemos usar para simular uma chamada por referência.

Quando queremos alterar as variáveis que são passadas para uma função, nós podemos declarar seus parâmetros formais como sendo ponteiros. Os ponteiros são a "referência" que precisamos para poder alterar a variável fora da função. O único inconveniente é que, quando usarmos a função, teremos de lembrar de colocar um & na frente das variáveis que estivermos passando para a função. Veja um exemplo:

```
#include <stdio.h>
void Troca (int *a,int *b);
void main (void)
{
      int num1,num2;
      num1=100;
      num2=200;
      Troca (&num1,&num2);
      printf ("\n\nEles agora valem %d  %d\n",num1,num2);
}

void Troca (int *a,int *b)
{
      int temp;
      temp=*a;
      *a=*b;
      *b=temp;
}
```

Não é muito difícil. O que está acontecendo é que passamos para a função Swap o endereço das variáveis num1 e num2. Estes endereços são copiados nos ponteiros a e b. Através do operador * estamos acessando o conteúdo apontado pelos ponteiros e modificando-o. Mas, quem é este conteúdo? Nada mais que os valores armazenados em num1 e num2, que, portanto, estão sendo modificados!

Espere um momento... será que nós já não vimos esta estória de chamar uma função com as variáveis precedidas de &? Já! É assim que nós chamamos a função scanf(). Mas porquê? Vamos pensar um pouco. A função scanf() usa chamada por referência porque ela precisa alterar as variáveis que passamos para ela! Não é para isto mesmo que ela é feita? Ela lê variáveis para nós e, portanto precisa alterar seus valores. Por isto passamos para a função o endereço da variável a ser modificada!

LISTA DE EXERCÍCIOS 02

A2_01) Faça uma função que recebe por parâmetro o raio de uma esfera e calcula o seu volume:

$$v = 4/3.PI .R^3$$

A2_02) Tendo como dados de entrada a altura e o sexo de uma pessoa, faça um programa que calcule seu peso ideal, utilizando as seguintes fórmulas:

- Para homens: (72.7*h) - 58
- Para mulheres: (62.1 *h) - 44.7 (h = altura)

A2_03) Faça uma função que recebe por parâmetro o tempo de duração de uma fábrica expressa em segundos e retorna também por parâmetro esse tempo em horas, minutos e segundos.

A2_04) Faça uma função que recebe a idade de uma pessoa em anos, meses e dias e retorna essa idade expressa em dias.

A2_05) Escreva um programa que retorne o número do quadrante (1,2,3 ou 4) através de uma função chamada VERIFICA_QUADRANTE, que deve receber um valor para x e um valor para y.

```
                Quadrantes
                      |
          2o          |        1o
                      |
---------------------------------------
                      |
          3o          |        4o
                      |
```

A2_06) Escreva um programa que calcule e retorne o salário atualizado através da função REAJUSTE. A função deve receber o valor do salário e o índice de reajuste.

A2_07) Escreva um programa que calcule e retorne por parâmetro o valor da hipotenusa através da função HIPOTENUSA. A função recebe o valor da base e da altura de um triângulo.

Fórmula: $hipotenusa^2 = base^2 + altura^2$

A2_08) Escreva um programa que calcule a expressão $X^2 + 4 * A * C - 36$ através de uma função chamada CALCULO. A função deve receber os valores de X, A e C. A função deve retornar o resultado desta expressão como parâmetro (não no retorno da função).